

Introduction to Programming (ITP)

Course Code: 101391 Language: C++

Concept Guide and Logic-Building Practice

How to use this workbook. Each guide gives the smallest useful implementation of a concept. The problems deliberately avoid naming the required algorithm. Read the story, identify the input and required output, write the decision rules on paper, and only then write C++ code. Problems within each topic increase in reasoning demand, but no difficulty labels are shown.

Contents

1 Arithmetic, Comparison, and Logical Operators

Arithmetic operators calculate values: +, -, *, /, and %. Comparison operators produce **true** or **false**: ==, !=, <, >, <=, and >=. Logical operators combine conditions: &&, ||, and !. Integer division discards the fractional part, and % gives the remainder.

```
int total = 47, boxSize = 6;
int fullBoxes = total / boxSize; // 7
int leftOver = total % boxSize; // 5
bool accepted = (fullBoxes >= 7) && (leftOver <= 5);
cout << fullBoxes << " " << leftOver << " " << accepted;
```

Problem 1

A courier receives a stack containing N sealed cards. The cards must be divided equally among K delivery riders. Every rider must receive the same whole number of cards. The dispatcher keeps any cards that cannot be divided equally. Display the cards received by one rider and the cards left behind. Then display whether the division was exact, using a comparison result. Use the entered values directly; do not add cards to the stack.

Implementation boundary: Use arithmetic and comparison operators as the main idea.

Problem 2

A vault uses a six-digit access number entered as an integer. The outer pair consists of the first and last digits. The vault opens when their sum equals the sum of the two middle digits. For 381246, the outer digits are 3 and 6, and the middle digits are 1 and 2. Extract only the required digits and report whether the rule holds. The access number is guaranteed to contain exactly six digits.

Implementation boundary: Use integer division, remainder, addition, and comparison.

Problem 3

A bridge sensor reports three Boolean signals: roadClear, windSafe, and override. Traffic may move when the road is clear and the wind is safe. An active override may also permit movement, but only when the road is clear. Construct one logical expression that represents the complete rule. Display the final result as 0 or 1 for every entered combination. Parentheses must make the intended priority unambiguous. Do not solve the cases separately.

Implementation boundary: Use one combined logical expression; do not use a selection statement.

Problem 4

A four-digit lock number $abcd$ is called balanced when $a + d = b + c$, and guarded when neither outer digit is zero. A number is accepted only when it is both balanced and guarded. For example, 2413 is balanced because $2 + 3 = 4 + 1$. Read one guaranteed four-digit positive integer. Extract its digits and display the balanced, guarded, and accepted results. Do not convert the number to a string.

Implementation boundary: Use arithmetic, comparison, and logical operators only.

2 If, Else If, Nested If, and Else

Selection statements execute different code according to conditions. Put narrower or exceptional cases before wider cases in an `else if` ladder. A nested `if` is useful when a second decision is meaningful only after the first succeeds.

```
int score = 78;
if (score < 0 || score > 100) {
    cout << "Invalid";
} else if (score >= 80) {
    cout << "Excellent";
} else if (score >= 60) {
    cout << "Pass";
} else {
    cout << "Retry";
}
```

Problem 1

A mountain tunnel receives a temperature reading in degrees Celsius. Below 0, the tunnel is closed because of ice. From 0 through 35, ordinary vehicles may enter. Above 35, only emergency vehicles may enter. Read the temperature and print exactly one operating instruction. The boundary values 0 and 35 belong to ordinary operation. No invalid temperature range needs to be checked.

Implementation boundary: Use one if-else-if ladder.

Problem 2

A research gate first checks whether a visitor owns a valid pass. Only a valid pass allows the gate to inspect the visitor's clearance level. Clearance 4 or above opens the laboratory; lower clearance opens the lobby. An invalid pass must immediately produce "Access Denied". Read the pass status and clearance level. Make the program reflect the order in which the real gate performs checks. Do not repeat the pass condition.

Implementation boundary: Use a nested decision.

Problem 3

A rescue drone has battery percentage B , wind value W , and payload P . It must stay grounded when $B < 25$, regardless of all other readings. Otherwise, a heavy payload above 8 may fly only when $W < 20$. A lighter payload may fly when $W < 35$. At an exact wind boundary, the flight is not permitted. Print "Launch", "Delay", or "Recharge" as appropriate. Arrange the decisions so that the emergency rule cannot be bypassed.

Implementation boundary: Use selection statements; compute no score or weighted average.

Problem 4

A scholarship office reads marks in Mathematics, Programming, and English. A student is ineligible if any mark is below 50. Among eligible students, "Distinction" requires an average of at least 80 and Programming marks of at least 85. Other eligible students receive "Qualified". Reject any individual mark outside the range 0 to 100 before classification. Print exactly one final message.

Implementation boundary: Use an ordered combination of if, else if, and nested if.

3 Ternary and Nested Ternary Operators

The ternary operator chooses one of two expressions. Its form is `condition ? valueIfTrue : valueIfFalse`. A nested ternary can classify several ordered cases, but parentheses should make the grouping clear.

```
int age = 17;
string access = (age >= 18) ? "Allowed" : "Not allowed";
int score = 74;
string grade = (score >= 80) ? "A" : ((score >= 60) ? "B" : "C");
cout << access << " " << grade;
```

Problem 1

A parcel scanner reads the actual weight and the permitted weight. It must print “Accept” when the parcel is within the limit and “Reject” when the parcel exceeds the limit. A parcel exactly equal to the limit is accepted. Store the chosen message in one string variable before printing it. Do not write separate output statements for the two outcomes. Only one comparison is necessary.

Implementation boundary: Use one ternary expression.

Problem 2

A signal tower receives an integer strength from 0 to 100. Values 70 or above are “Strong”; values 30 through 69 are “Weak”. Values below 30 are “Lost”. Read the strength and assign exactly one label. The order of tests must handle 30 and 70 correctly. Print the assigned label once. The input is guaranteed valid.

Implementation boundary: Use a nested ternary expression, not an if statement.

Problem 3

A navigation device compares distances to three charging stations: A, B, and C. It must display the letter of a nearest station. When A ties with another station, prefer A. When only B and C tie, prefer B. Read three positive distances and select the result in one assignment. Do not rearrange the distances or calculate differences. Print only the chosen station letter.

Implementation boundary: Use comparison operators inside nested ternary expressions.

Problem 4

A coded dial contains an integer from 0 to 999. The system labels it “Mirror” when the first and last digits match. Otherwise, it labels the larger outer digit as “Left” or “Right”. For a one-digit number, the missing hundreds digit is treated as zero. Extract both outer digits arithmetically. Assign the complete label through one expression and print it once. No selection statement may be used.

Implementation boundary: Use arithmetic plus a nested ternary expression.

4 Output Alignment with `setw`, `left`, and `right`

Include `<iomanip>`. The manipulator `setw(n)` sets the minimum width of the *next* output item only. `left` and `right` remain active until changed.

```
#include <iomanip>
cout << left << setw(14) << "Item"
      << right << setw(8) << "Price" << '\n';
cout << left << setw(14) << "Notebook"
      << right << setw(8) << 250 << '\n';
```

Problem 1

A tiny library prints one borrowing receipt. Read a book title without spaces, a shelf code, and the number of days borrowed. Display the headings Book, Shelf, and Days on one line. Display the three entered values directly below their headings. Book and Shelf must be left aligned; Days must be right aligned. Use column widths 18, 10, and 6 respectively. Both rows must follow exactly the same layout.

Implementation boundary: Use setw, left, and right; do not insert alignment spaces manually.

Problem 2

A race controller displays two runners and their finish times. Each name occupies a left-aligned field of width 16. Each time occupies a right-aligned field of width 10. After reading both records, print a heading row and two data rows. Add a vertical bar between the fields without breaking alignment. Names contain no spaces, and times are decimal values. The decimal precision does not need to be changed.

Implementation boundary: Build the layout entirely with stream manipulators.

Problem 3

A warehouse label contains a product code, current stock, and reorder level. The code must be left aligned in width 15. Both numbers must be right aligned in fields of width 9. Print one heading row, one separator row made from fixed text, and one data row so that each value ends under its heading. A product code may have from 1 to 14 characters. The output must remain aligned for all valid codes.

Implementation boundary: Use repeated setw calls and deliberate changes of alignment.

Problem 4

A scoreboard prints three team names with their goals and penalties. A team field has width 14; each numeric field has width 8. Text is left aligned and every number is right aligned. Print a heading followed by exactly three records read from input. No loop is required for these three records. Changing one name from 3 to 13 characters must not move numeric columns. Use a single formatting pattern for every record.

Implementation boundary: Use setw correctly for every individual field.

5 Implicit and Explicit Type Casting

Implicit casting happens automatically when C++ converts a value to a compatible wider type. Explicit casting requests a conversion using `static_cast<type>(value)`. Cast before integer division when a fractional answer is required.

```
int total = 17, count = 4;
double average = static_cast<double>(total) / count; // 4.25
double price = 99.80;
```

```
int wholePart = static_cast<int>(price);           // 99
cout << average << " " << wholePart;
```

Problem 1

A runner completes 7 laps in a total number of seconds entered by the user. The coach wants the average seconds per lap as a decimal value. Both stored values are integers. Calculate and display the average without losing the fractional part. For 100 seconds, the result must not become 14. Do not change the declared types of the original inputs. Convert only where the calculation requires it.

Implementation boundary: Use explicit casting before division.

Problem 2

A fuel pump records delivered fuel as a decimal number of litres. Its mechanical counter stores only complete litres. Read the decimal measurement and display the counter's stored integer value. Then display the unrecorded fractional amount. For 18.75 litres, the counter stores 18 and leaves 0.75. Do not round the measurement. The input is nonnegative.

Implementation boundary: Use explicit conversion to obtain the whole part.

Problem 3

A sensor stores a character representing one digit from '0' to '9'. The controller needs the corresponding numeric digit for arithmetic. Read the character and convert it by using its relationship with '0'. Then display the square of the numeric digit. For character '7', the program must calculate 49, not a character code. Do not use string conversion functions. Explain through the expression where implicit conversion occurs.

Implementation boundary: Use character arithmetic and type conversion behavior.

Problem 4

A voting machine records Y yes votes and T total votes as integers. It must display the approval percentage, including any fractional part. It must also store a whole-number percentage by discarding, not rounding, decimals. For $Y = 2$ and $T = 3$, the precise result is about 66.67 while the stored whole result is 66. Assume $T > 0$ and preserve both results in separate variables. The original integer inputs must remain unchanged.

Implementation boundary: Use explicit casting at the correct stages of the calculation.

6 Single For Loop

A **for** loop groups initialization, continuation, and update. The body may maintain several pieces of state. The problems in this section permit exactly one non-nested **for** loop; previously practised expressions and decisions may appear inside it.

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    int value;
    cin >> value;
    if (value > 0) sum += value;
}
cout << sum;
```

Problem 1

A radio archive stores a decimal identifier as text. It contains at most 10 digits before and 10 digits after one decimal point. Count the digits on the left and on the right of the point. Signs are not used, and the point is guaranteed to appear exactly once. For 2048.007, report 4 digits on the left and 3 on the right. Inspect each character once from left to right. Do not convert the identifier to a numeric type.

Implementation boundary: Use one simple for loop, indexing, and a state that changes at the point.

Problem 2

A transmission arrives as a sequence of decimal digits without spaces. A digit is called a peak when it is greater than both immediate neighbours. The first and last digits can never be peaks. For 352943, the digits 5 and 9 are peaks. Count and display all peaks in the received sequence. The sequence contains at least three digits. Each eligible position should be examined exactly once.

Implementation boundary: Use one for loop and direct character indexing.

Problem 3

A bank checks a numeric message using an alternating checksum. Starting from the left, add the first digit, subtract the second, add the third, subtract the fourth, and continue in this pattern. For 5724, the checksum is $5 - 7 + 2 - 4 = -4$. Read up to 20 digits as a string and display the final checksum. The message contains digits only. Do not create separate collections for odd and even positions.

Implementation boundary: Use one for loop and update one running total.

Problem 4

A calibration device approximates $\sqrt{S}$, where $S > 0$. Begin with a search interval from 0 to $\max(1, S)$. During each step, inspect the midpoint: if its square is too small, replace the lower end; otherwise replace the upper end. Perform exactly 25 refinement steps and display the final midpoint. The loop must not test candidate values one by one. No library square-root function may be called.

Implementation boundary: Use one for loop implementing interval bisection.

7 Switch Statements

A **switch** selects a branch using an integral or character expression. A **break** prevents execution from falling into the next case. Several case labels may intentionally share one action.

```
char command;
cin >> command;
switch (command) {
    case 'S': case 's': cout << "Start"; break;
    case 'P': case 'p': cout << "Pause"; break;
    default:          cout << "Unknown";
}
```

Problem 1

A lift panel receives a floor code: B, G, 1, 2, or 3. Each valid code displays its destination name. B means Basement and G means Ground Floor. The numeric characters display First, Second, or Third Floor. Both lowercase and uppercase B and G must work. Any other character prints “Invalid Floor”. Read only one character.

Implementation boundary: Use a switch, shared cases where useful, and a default branch.

Problem 2

A small calculator reads an integer, an operator character, and another integer. Supported operators are +, -, *, /, and %. Select the required calculation from the operator. Division and remainder by zero must produce “Undefined”. An unsupported operator must produce “Invalid Operator”. Print only one result or one error message. The operands themselves must not be changed.

Implementation boundary: Use switch for operator selection, with a local safety check where required.

Problem 3

A railway kiosk encodes a travel zone from 1 through 5. Zones 1 and 2 cost 50, zones 3 and 4 cost 80, and zone 5 costs 120. The passenger also enters whether a return trip is required. A return trip costs exactly twice the selected one-way fare. Invalid zones must not be assigned a fare. Share case logic wherever two zones have the same cost. Display either the final fare or “Invalid Zone”.

Implementation boundary: Use one switch statement; do not write five separate if conditions.

Problem 4

A date terminal receives a month number from 1 to 12 and a year. Display the number of days in that month. February has 29 days when the year is divisible by 400, or when divisible by 4 but not by 100; otherwise it has 28. Group all 30-day months and all 31-day months appropriately. Reject month numbers outside the valid range. Print exactly one answer.

Implementation boundary: Use switch for the month and a decision only inside February’s case.

8 Increment and Decrement Operators

Prefix changes the variable before its value is used; postfix yields the old value and then changes the variable. Avoid combining several changes to the same variable in one complicated expression.

```
int ticket = 40;
cout << ticket++ << '\n'; // prints 40, ticket becomes 41
cout << ++ticket << '\n'; // ticket becomes 42, prints 42
cout << ticket-- << '\n'; // prints 42, ticket becomes 41
```

Problem 1

A ticket dispenser stores the next available number as 301. A customer must see the current number before the dispenser advances. Print the assigned ticket by using the stored value once. Then print the number now held for the next customer. Do not add the number 1 explicitly. The first printed ticket must remain 301. Only one variable is needed.

Implementation boundary: Use postfix increment and observe its order of effect.

Problem 2

A countdown panel stores 10 remaining attempts. Before displaying the new count, one failed attempt must be removed. Print the updated value in the same statement that performs the change. Then restore one attempt and print the restored value, again changing and displaying it together. The required output values are 9 and then 10. Do not use addition or subtraction assignment.

Implementation boundary: Choose prefix decrement and prefix increment deliberately.

Problem 3

A queue controller stores `front=7` and `back=12`. It first prints and consumes the current front position. It then creates a new position beyond the back and prints that new position. Finally, display the two values now stored by the controller. The consumed number must be printed before front changes; the new back number must be printed after back changes. No arithmetic operator other than increment may be used.

Implementation boundary: Use postfix and prefix increment according to timing.

Problem 4

A game checkpoint stores `level=5` and `lives=3`. A level completion must first raise the level, then display the new level. A trap must display the current lives before removing one. After both events, print the final stored state. Write each event as a separate, easy-to-read output statement. Do not hide multiple modifications in one expression. The visible event values and final values must reflect the timing rules.

Implementation boundary: Demonstrate the semantic difference between prefix and postfix.

9 Random Number Generation

The traditional introductory approach seeds once with `srand` and obtains a value from `rand`. The expression `low + rand() % (high-low+1)` maps the result to an inclusive range. Seed once near the beginning, never before every draw.

```
#include <cstdlib>
#include <ctime>
srand(static_cast<unsigned>(time(0)));
int die = 1 + rand() % 6;           // 1 through 6
int token = 1000 + rand() % 9000; // 1000 through 9999
cout << die << " " << token;
```

Problem 1

A weather simulator needs one integer wind reading from 15 through 35. Generate the reading so that both boundary values are possible. Display the generated number followed by “km/h”. Seed the generator once when the program starts. Do not reject and regenerate values outside the range. The range must be produced directly by one expression. Repeated program runs should not always begin with the same reading.

Implementation boundary: Use `srand`, `time`, `rand`, `remainder`, and an `offset`.

Problem 2

A lock creates a three-digit security code from 000 through 999. The numeric value may therefore be smaller than 100. Generate one value in the correct range. Print it as three separate digits so that leading zeros remain visible. For generated value 7, the display must be 007. Extract the hundreds, tens, and units positions arithmetically. Do not regenerate small values.

Implementation boundary: Use one random value plus division and remainder.

Problem 3

A game spinner has unequal labels but equal numbered sectors 0 through 9. Sectors 0–4 award “Copper”, 5–7 award “Silver”, and 8–9 award “Gold”. Generate one sector and display both its number and reward. Every sector must be equally reachable. Seed only once, before generating the sector. Do not generate a reward category independently.

Implementation boundary: Generate one random digit and derive the outcome from it.

Problem 4

A training console creates two random digits a and b , each from 0 to 9. It displays the larger digit first to form a two-digit challenge number. If the digits are equal, their order naturally remains unchanged. Generate each digit exactly once and preserve both generated values. For digits 2 and 8, the challenge shown must be 82. Do not generate values repeatedly until an order appears. Display the digits without adding an extra space between them.

Implementation boundary: Use two random generations and comparison logic.

10 ANSI Terminal Colors and Resetting

ANSI escape sequences begin with `\033[` and end with `m`. Print a color code before text and **always** print reset code `\033[0m` afterward so later output returns to normal.

30	Black	31	Red
32	Green	33	Yellow
34	Blue	35	Magenta
36	Cyan	37	White
1	Bold/bright	0	Reset

```
cout << "\033[31m" << "Error" << "\033[0m\n";
cout << "\033[32m" << "Success" << "\033[0m\n";
cout << "\033[1;36m" << "Bright cyan" << "\033[0m\n";
// Background examples: 41 red, 42 green, 43 yellow, 44 blue.
```

Problem 1

A maintenance console prints “SYSTEM READY” in green. The following text, “Normal output restored”, must use the default color. Display both messages on separate lines. Apply the color before the first message only. Reset immediately after that message. Do not color the second line accidentally. Use the numeric ANSI values rather than a library.

Implementation boundary: Use green code 32 and reset code 0.

Problem 2

A battery terminal reads a percentage from 0 to 100. Print the percentage in green when it is at least 60, yellow from 30 through 59, and red below 30. The percent sign must have the same color as the number. After the reading, print “ checked” in the default terminal color. Boundary values 30 and 60 must enter the correct bands. Reset exactly after the colored reading.

Implementation boundary: Select among ANSI 32, 33, and 31, then use ANSI 0.

Problem 3

A security panel reads two Boolean values: alarm and doorOpen. When alarm is active, print bold white text on a red background. Otherwise, an open door is yellow and a closed door is green. Use `\033[1;37;41m` for the alarm style. Display exactly one status message according to priority. Reset the terminal after the selected message. An active alarm must override the door state.

Implementation boundary: Combine conditional selection with explicit ANSI sequences.

Problem 4

A heat display reads an integer from 0 to 999 and prints its three digits. Each digit has its own severity color: 0–3 green, 4–6 yellow, 7–9 red. For 582, the first digit is yellow, the second red, and the third green. Print the digits next to one another with no spaces. Choose and apply a color separately before each digit. Reset once after the complete three-digit display. The input is guaranteed to contain exactly three digits.

Implementation boundary: Use digit extraction, decisions, ANSI colors, and one final reset; no loop.

11 Strings: Concatenation, Indexing, and Length

A `string` stores a sequence of characters. Join strings with `+`, access positions from 0 using `[]`, and obtain the number of characters with `length()`. For a nonempty string `s`, the last character is `s[s.length()-1]`.

```
#include <string>
string first = "Ali", last = "Khan";
string full = first + " " + last;
char initial = full[0];
char finalCharacter = full[full.length() - 1];
cout << full << " " << initial << " " << finalCharacter;
```

Problem 1

A conference creates a badge from a first name and a city code. Read both as strings without spaces. The visible badge must contain the first name, a hyphen, and the city code. Also print the first character of the name and the last character of the code. The two inputs are guaranteed nonempty. Build the badge in one separate string variable. Do not print its pieces one at a time.

Implementation boundary: Use concatenation, indexing, and length.

Problem 2

A parcel tag is valid when its first and last characters are the same, and its two central characters are also the same. Every tag is guaranteed to have an even length of at least four. For length n , the central positions are $n/2 - 1$ and $n/2$. Read one tag and inspect only the four required positions. Display “Linked” when both rules hold and “Broken” otherwise. Do not examine the tag with a loop.

Implementation boundary: Use length, indexing, comparison, and logical operators.

Problem 3

A company forms an identifier from a nonempty first name and last name. The identifier consists of the first letter of the first name, the last letter of the last name, and the total number of letters. For **Sara Khan**, the character part is **Sn** and the count is 8. Read the names without spaces and compute the total length. Display the character part and count together as one identifier. No loop or string-search function is allowed.

Implementation boundary: Use indexing, length, and concatenation/output composition.

Problem 4

A six-character bridge code is split conceptually into two halves. It is accepted when character 0 matches character 5, character 1 matches character 4, and character 2 matches character 3. Read a code and first reject it if its length is not exactly six. Otherwise evaluate all three mirror links. Display only “Accepted” or “Rejected”. Do not reverse the code and do not use a loop.

Implementation boundary: Use length, direct indexing, and one combined logical test.

Instructor Note

The implementation boundaries intentionally restrict students to the covered material. They may use earlier practised concepts to support the named topic, but should not replace the intended idea with arrays, nested loops, library algorithms, search functions, or other unintroduced features.